Reporte Técnico de Actividades Práctico-Experimentales Nro 3.

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Ivett Zaragocin
Asignatura	Simulación
Ciclo	5
Unidad	1
Resultado de aprendizaje de la unidad	**R1.** Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Simulación de un sistema de líneas de espera M/M/1 mediante SimPy
Nombre del Docente	José O. Guamán Q.
Fecha	15 de Octubre de 2026 16 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Simular un sistema de líneas de espera M/M/1 utilizando Python y SimPy, analizando el comportamiento de los clientes, la cola y el servidor.

- Implementar un modelo M/M/1 utilizando la biblioteca SimPy.
- Calcular las principales métricas teóricas del sistema de espera.
- Comparar los resultados teóricos con los obtenidos mediante la simulación

3. Materiales, Reactivos, Equipos y Herramientas

- Python 3.x.
- Biblioteca SimPy.
- NumPy.
- Matplotlib.
- Editor de código: Visual Studio Code, PyCharm o Jupyter Notebook.

- Computador.
- Procesador de 2 núcleos o superior.
- Mínimo 4 GB de RAM.
- Sistema operativo Linux o macOS

4. Procedimiento / Metodología Ejecutada

Fundamento teórico

Un sistema de líneas de espera está formado por clientes que llegan a solicitar un servicio, una cola en la que esperan y uno o más servidores que los atienden [1]. En la notación de Kendall, el sistema M/M/1 tiene tiempos entre llegadas exponenciales (primera M), tiempos de servicio exponenciales (segunda M) y un único servidor (1). En este trabajo la disciplina de atención es FIFO, es decir, el primer cliente en llegar es el primero en ser atendido, y la cola no tiene límite de capacidad [1], [2].

La letra M significa «markoviano». Las llegadas forman un proceso de Poisson con tasa λ, por lo que el tiempo entre dos llegadas consecutivas sigue una distribución exponencial con media $1/\lambda$; del mismo modo, la atención de cada cliente dura un tiempo exponencial con media $1/\mu$ [2]. La distribución exponencial carece de memoria: la probabilidad de que un evento ocurra en el instante siguiente no depende del tiempo que ya transcurrió, y gracias a esa propiedad el sistema puede modelarse como un proceso de Markov [3].

El parámetro central del modelo es la intensidad de tráfico o utilización del servidor, que representa la fracción del tiempo en que el servidor permanece ocupado [2]. El sistema alcanza un estado estacionario solo si la utilización es menor que 1, es decir, si los clientes llegan más despacio de lo que el servidor puede atenderlos; en caso contrario la cola crece sin límite [1]:

$$\rho = \lambda / \mu < 1 \quad (1)$$

En estado estacionario, las medidas de desempeño del modelo M/M/1 se obtienen con las siguientes expresiones [1], [2]:

$$L = \lambda / (\mu - \lambda) \quad (2)\ \textit{clientes promedio en el sistema}$$

$$Lq = \lambda^2 / (\mu(\mu - \lambda)) \quad (3)\ \textit{clientes promedio en la cola}$$

$$W = 1 / (\mu - \lambda) \quad (4)\ \textit{tiempo promedio en el sistema}$$

$$Wq = \lambda / (\mu(\mu - \lambda)) \quad (5)\ \textit{tiempo promedio de espera en la cola}$$

Estas medidas se relacionan mediante la ley de Little, $L = \lambda W$ y $Lq = \lambda Wq$, que se cumple en cualquier sistema de colas estable [4]. Además, $W = Wq + 1/\mu$, porque el tiempo total que un cliente pasa en el sistema es su espera en la cola más su tiempo de servicio.

Para estudiar el sistema sin depender únicamente de las fórmulas se utiliza la simulación de eventos discretos: el estado del sistema cambia solo en instantes concretos, como una llegada o el fin de un servicio, y el reloj de la simulación avanza de un evento al siguiente [3]. SimPy es una biblioteca de Python para este tipo de simulación basada en procesos: cada cliente se representa como un proceso que espera eventos y el servidor como un recurso de capacidad limitada [5].

Finalmente, para estimar el tamaño promedio de la cola a partir de la simulación puede usarse lo que observa cada cliente al llegar, ya que las llegadas de Poisson

observan el sistema en su promedio temporal (propiedad PASTA, Poisson Arrivals See Time Averages) [6].

La tabla 1 resume la notación utilizada en este informe.

Símbolo	Significado	Expresión
λ	Tasa promedio de llegada de clientes	clientes / unidad de tiempo
μ	Tasa promedio de servicio del servidor	clientes / unidad de tiempo
ρ	Utilización del servidor	λ / μ
L	Número promedio de clientes en el sistema	λ / (μ − λ)
Lq	Número promedio de clientes en la cola	λ² / (μ(μ − λ))
W	Tiempo promedio de un cliente en el sistema	1 / (μ − λ)
Wq	Tiempo promedio de espera en la cola	λ / (μ(μ − λ))

Tabla 1. Notación del modelo M/M/1

Metodología ejecutada

La práctica se desarrolló en Python 3.12 con SimPy, NumPy y Matplotlib, siguiendo estas etapas:

- **Análisis de la guía.** Se revisaron el modelo M/M/1, sus fórmulas, el pseudocódigo y los tres escenarios propuestos en la guía de la práctica [7]. En este paso se detectó que la fórmula de la utilización debía ser ρ = λ/μ.
- **Creación del repositorio.** Se creó el repositorio en GitHub (github.com/ivecanahi/ape3) con la estructura base del proyecto: carpetas src/, tests/ y resultados/, y el archivo requirements.txt.
- **División del trabajo por módulos.** El proyecto se dividió en cuatro partes independientes (teoría, simulación, gráficas e integración). Antes de programar se acordaron los nombres de las funciones y de los datos que cada parte entrega, para poder avanzar en paralelo.
- **Trabajo por ramas.** Cada parte se desarrolló en una rama propia de Git y se integró a la rama principal (main) mediante Pull Requests.
- **Desarrollo con pruebas.** Las fórmulas teóricas se programaron escribiendo primero las pruebas con los valores esperados de cada escenario y luego el código. En total, el proyecto tiene 17 pruebas automatizadas (pytest).
- **Integración.** Se unieron la teoría, la simulación y las gráficas en un programa principal (main.py) que ejecuta los tres escenarios con un solo comando.
- **Revisión y corrección.** Al revisar el repositorio se detectó que el tamaño de la cola solo se registraba cuando llegaba un cliente. Se corrigió para registrarlo también cuando un cliente pasa a ser atendido y se regeneraron las gráficas. También se mejoró la presentación de los resultados en la consola.
- **Ejecución y análisis.** Se ejecutaron los tres escenarios, se compararon los resultados teóricos con los simulados y se analizaron las gráficas.

El proyecto quedó organizado en los siguientes módulos:

Módulo	Función
src/teoria.py	Calcula ρ, L, Lq, W y Wq y valida la estabilidad (λ < μ).
src/simulacion.py	Simula la cola M/M/1 con SimPy y registra los tiempos de espera y el tamaño de la cola.
src/graficas.py	Genera las gráficas de la cola, de los tiempos de espera y la comparación.
src/experimentos.py	Define los escenarios, une teoría y simulación y genera la tabla comparativa.
src/consola.py	Presenta los resultados en la consola con formato (biblioteca rich).
main.py	Programa principal: ejecuta todo con el comando python main.py.

Tabla 2. Módulos del proyecto

La simulación, que sigue el pseudocódigo de la guía, funciona así:

- Se definieron λ, μ, el tiempo de simulación (10 000 unidades de tiempo) y una semilla fija (42) para que los resultados sean reproducibles.
- Antes de simular se verificó la condición de estabilidad λ < μ; si no se cumple, el programa muestra el mensaje «El sistema no es estable».
- Se creó el entorno de SimPy (simpy.Environment) y un servidor con capacidad 1 (simpy.Resource) [5].
- Un proceso generador crea clientes con tiempos entre llegadas exponenciales de media 1/λ.
- Cada cliente registra su llegada, solicita el servidor, espera su turno en orden de llegada (FIFO), registra su tiempo de espera y es atendido durante un tiempo exponencial de media 1/μ; al terminar libera el servidor.
- El tamaño de la cola se registró cuando llega cada cliente y cuando cada cliente pasa a ser atendido. El tamaño promedio de la cola simulado es el promedio de la cola observada por los clientes al llegar; por la propiedad PASTA (las llegadas de Poisson observan el promedio temporal del sistema), es un estimador válido de Lq [6].
- Al finalizar se calcularon los promedios simulados, las métricas teóricas y el error porcentual |teórico − simulado| / teórico × 100, y se generaron la tabla comparativa y las gráficas.

Se ejecutaron los tres escenarios propuestos en la guía:

Escenario	λ	μ	ρ = λ/μ	Condición
1	2.0	3.0	0.667	Sistema estable
2	2.5	3.0	0.833	Mayor utilización
3	2.9	3.0	0.967	Cerca de saturación

Tabla 3. Escenarios de simulación

Para reproducir los resultados se ejecuta *pip install -r requirements.txt* y luego *python main.py*. El programa imprime los resultados en la consola y guarda las gráficas y la tabla en la carpeta *resultados/*.

5. Resultados

La tabla 4 presenta las métricas teóricas de cada escenario y la tabla 5 las compara con los resultados de la simulación (tiempo de simulación = 10 000; semilla = 42).

Escenario	λ	μ	ρ	L	Lq	W	Wq
1	2.0	3.0	0.667	2.000	1.333	1.000	0.667
2	2.5	3.0	0.833	5.000	4.167	2.000	1.667
3	2.9	3.0	0.967	29.000	28.033	10.000	9.667

Tabla 4. Métricas teóricas del modelo M/M/1

Esc.	Clientes atendidos	Wq teórico	Wq simulado	Error Wq	Lq teórico	Lq simulado	Error Lq
1	19 997	0.667	0.705	5.8 %	1.333	1.398	4.8 %
2	25 008	1.667	1.799	7.9 %	4.167	4.490	7.8 %
3	28 906	9.667	8.228	14.9 %	28.033	23.758	15.2 %

Tabla 5. Comparación entre resultados teóricos y simulados

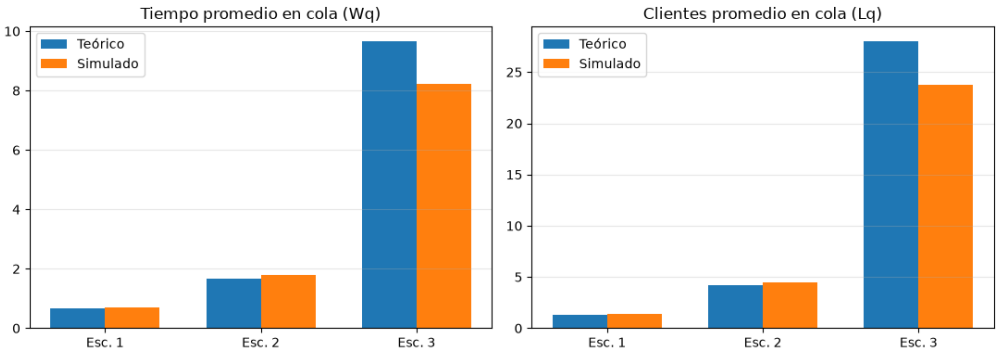


Figura 1. Comparación teórico vs. simulado de Wq y Lq en los tres escenarios

Comportamiento de la cola

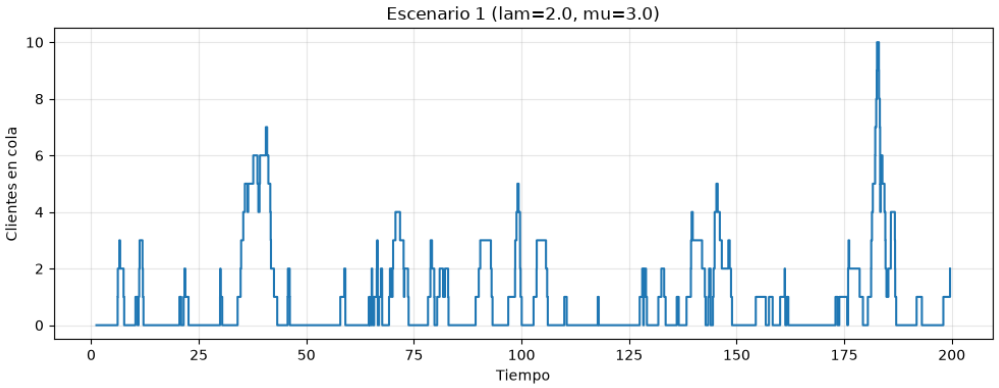


Figura 2. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 1

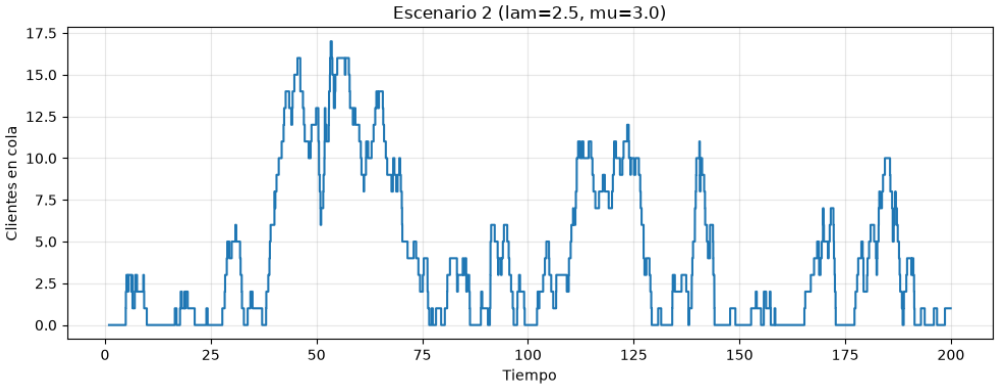


Figura 3. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 2

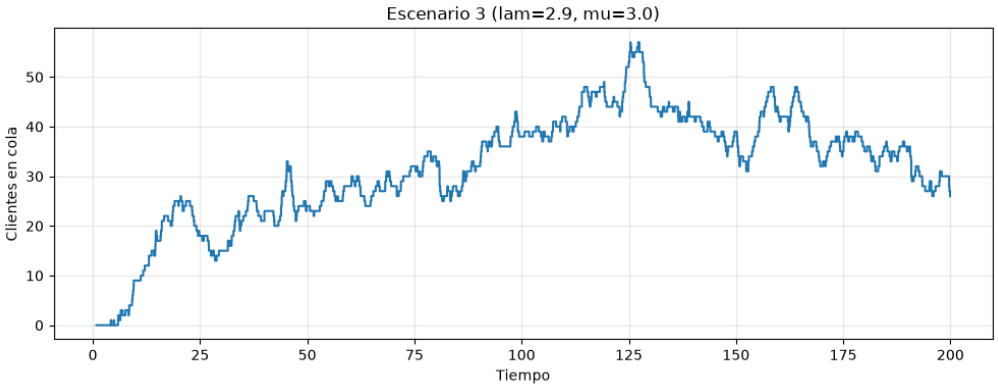


Figura 4. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 3

Distribución de los tiempos de espera

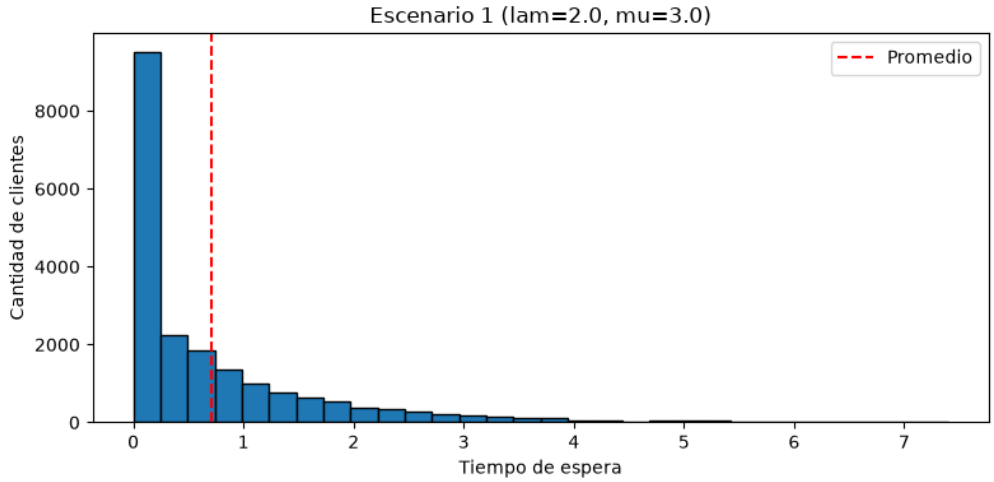


Figura 5. Histograma de tiempos de espera, escenario 1

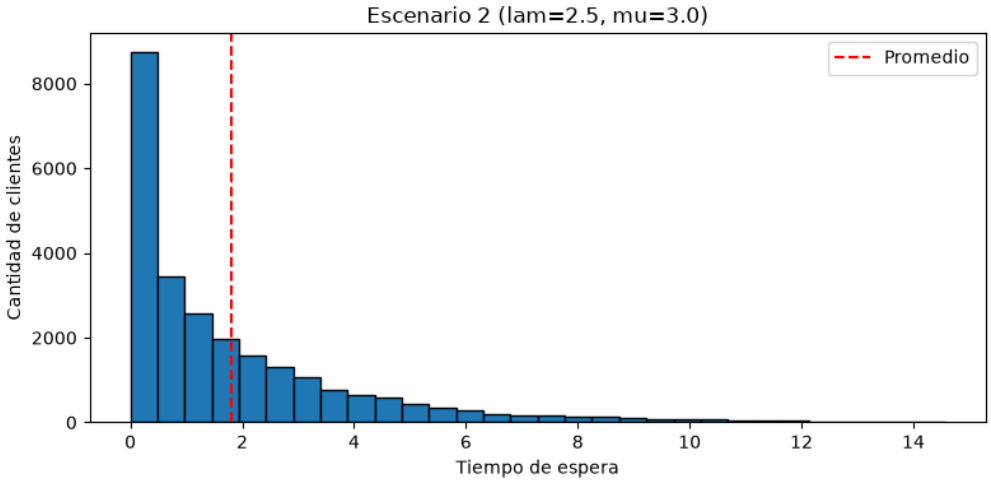


Figura 6. Histograma de tiempos de espera, escenario 2

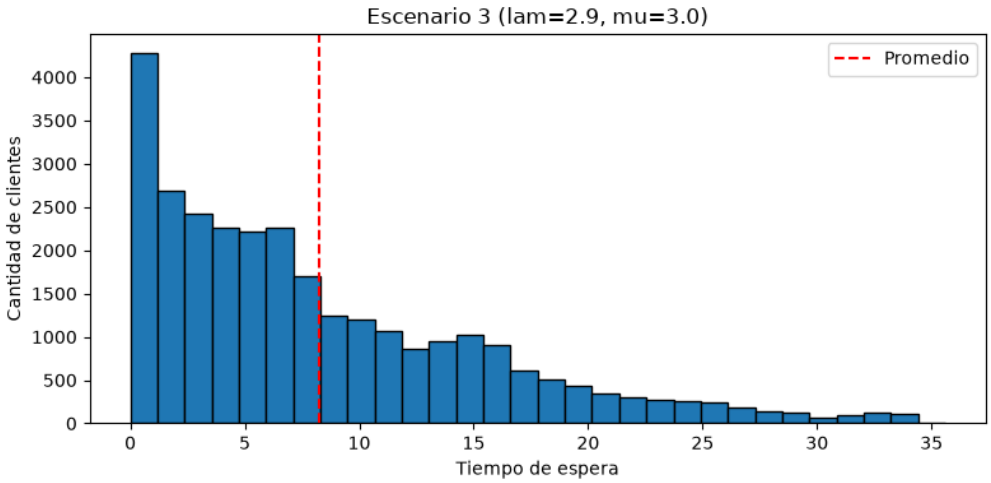


Figura 7. Histograma de tiempos de espera, escenario 3

Análisis de resultados

- Al aumentar λ de 2.0 a 2.9 con μ = 3.0, la utilización del servidor sube de 66.7 % a 96.7 %, pero el tiempo promedio de espera en cola teórico pasa de 0.667 a 9.667 unidades de tiempo (14.5 veces más) y la cola promedio de 1.33 a 28.03 clientes. El crecimiento no es lineal, porque el término (μ − λ) del denominador de las fórmulas se acerca a cero.
- En los escenarios 1 y 2 la simulación coincide con la teoría con errores menores al 8 %, lo que valida el modelo implementado (figura 1).
- En el escenario 3 el error sube a cerca del 15 % y la simulación subestima la espera (8.228 frente a 9.667). Cerca de la saturación la cola tiene una variabilidad muy alta y tarda mucho en alcanzar el estado estacionario; además, la simulación comienza con el sistema vacío y se realizó una sola corrida [3].
- En el escenario 1 la cola se vacía con frecuencia y sus picos son cortos (figura 2). En el escenario 3 la cola nunca llega a vaciarse después de los primeros instantes: crece durante las primeras unidades de tiempo y luego oscila entre 20 y 55 clientes (figura 4). Esa subida inicial muestra el período de calentamiento que explica parte del error.

- Los histogramas de espera (figuras 5 a 7) tienen forma exponencial decreciente: la mayoría de los clientes espera poco y pocos esperan mucho. En el escenario 1 la barra de espera casi nula es la más alta porque, con ρ = 0.667, cerca de un tercio de los clientes encuentra el servidor libre.

6. Preguntas de Control

- **¿Qué representa el modelo M/M/1?**

  El modelo M/M/1 representa un sistema de líneas de espera o llamadas colas que se pueden caracterizar por:

  - Llegadas de clientes según un proceso de Poisson (primera M).
  - Tiempos de servicio con distribución exponencial (segunda M).
  - Un único servidor (el 1).
  - Atención en orden de llegada (FIFO).

- **¿Qué representan λ y μ?**

  λ(lambda): Es la tasa promedio de la llegada de clientes por unidad de tiempo

  μ(mu): Es la tasa promedio de servicio del servidor o la capacidad de atención por unidad de tiempo

- **¿Por qué debe cumplirse λ < μ para que el sistema sea estable?**

  Esta condición debe cumplirse porque la tasa con la que llegan los clientes tiene que ser menor que la capacidad de atención del servidor. Si λ ≥ μ, entonces ρ ≥ 1: el servidor no alcanza a atender a todos los clientes y la cola crece sin límite [1].

- **¿Qué significa la utilización ρ?**

  La utilización ρ = λ / μ indica la proporción del tiempo que el servidor pasa ocupado atendiendo clientes. Por ejemplo, en el escenario 1, ρ = 0.667 significa que el servidor está ocupado el 66.7 % del tiempo.

- **¿Cuál es la diferencia entre W y Wq?**

  La diferencia entre **W** y **Wq** es que W es el tiempo promedio que un cliente permanece en todo el sistema ya sea en la cola como el tiempo que dura su atención, mientras que Wq es el tiempo que el cliente pasa esperando en una cola hasta que este sea atendido.

- **¿Cuál es la diferencia entre L y Lq?**

  La diferencia que tienen **L** y **Lq** es que L es el número promedio de clientes que se encuentran en todo el sistema incluyendo los que están en cola y el que esté siendo atendido, mientras que Lq es el número promedio de clientes que se encuentran en una única cola a espera de su turno.

- **¿Por qué SimPy utiliza una distribución exponencial para generar los tiempos?**

SimPy no impone una distribución: en esta práctica se usa la distribución exponencial porque el modelo M/M/1 lo exige. Si las llegadas siguen un proceso de Poisson con tasa λ, los tiempos entre llegadas son exponenciales con media 1/λ, y los tiempos de servicio son exponenciales con media 1/μ. La exponencial tiene la propiedad de falta de memoria, que convierte al sistema en un proceso de Markov y permite aplicar las fórmulas teóricas [2], [3].

- **¿Qué sucede con la cola cuando λ se aproxima a μ?**

  La cola y el tiempo de espera aumentan de forma no lineal, porque el servidor trabaja cerca de su capacidad máxima y no tiene margen para recuperarse de una racha de llegadas. En la práctica, al pasar de λ = 2.5 a λ = 2.9, Wq teórico pasa de 1.667 a 9.667 (5.8 veces más).

- **¿Por qué los resultados simulados pueden ser diferentes de los resultados teóricos?**

  Los resultados teóricos son valores exactos del estado estacionario, es decir, del comportamiento del sistema después de un tiempo infinito. La simulación usa una muestra finita de clientes generada con números aleatorios, comienza con el sistema vacío y depende de la semilla. Por eso aparece un error, que crece cerca de la saturación (14.9 % en el escenario 3) y que disminuye al aumentar el tiempo de simulación o el número de corridas [3].

- **¿Qué ocurriría si λ fuera mayor que μ?**

  El sistema no sería estable: llegarían más clientes de los que el servidor puede atender, por lo que la cola y el tiempo de espera crecerían indefinidamente y las fórmulas teóricas dejarían de ser válidas (darían valores negativos). Por eso el programa valida λ < μ antes de simular.

7. Conclusiones

- Se implementó un modelo M/M/1 funcional en Python con SimPy, con llegadas de Poisson, tiempos de servicio exponenciales, un único servidor y disciplina FIFO.
- Las métricas simuladas coinciden con las teóricas con errores menores al 8 % en los escenarios 1 y 2, lo que valida el modelo de simulación.
- Cuando λ se aproxima a μ el sistema se degrada de forma no lineal: al subir la utilización de 83.3 % a 96.7 %, el tiempo de espera en cola teórico pasa de 1.667 a 9.667 unidades de tiempo.
- Cerca de la saturación la simulación necesita más tiempo o más corridas para aproximarse a la teoría: con una sola corrida, el error en el escenario 3 fue de aproximadamente 15 %.

8. Recomendaciones

- Ejecutar varias corridas independientes con distintas semillas y promediar los resultados, reportando intervalos de confianza.

- Descartar un período de calentamiento al inicio de la simulación para reducir el sesgo de comenzar con el sistema vacío, sobre todo cuando ρ es cercano a 1.
- En un sistema real, mantener la utilización del servidor con margen respecto a 1, porque cerca de la saturación pequeños aumentos de la demanda producen esperas muy largas.

9. Bibliografía / Referencias

[1] H. A. Taha, *Investigación de operaciones*, 9.ª ed. México: Pearson Educación, 2012.

[2] F. S. Hillier y G. J. Lieberman, *Introducción a la investigación de operaciones*, 9.ª ed. México: McGraw-Hill, 2010.

[3] J. Banks, J. S. Carson, B. L. Nelson y D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4] J. D. C. Little, "A proof for the queuing formula: L = λW," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[5] Team SimPy, "SimPy: Discrete event simulation for Python," *SimPy Documentation*. [En línea]. Disponible en: https://simpy.readthedocs.io/

[6] R. W. Wolff, "Poisson arrivals see time averages," *Operations Research*, vol. 30, no. 2, pp. 223–231, 1982.

[7] J. O. Guamán Q., "Guía de actividades práctico-experimentales Nro. 003: Simulación de un sistema de líneas de espera M/M/1 mediante SimPy," Carrera de Computación, Universidad Nacional de Loja, Loja, Ecuador, 2026.

10. Anexos

- **Repositorio:** **https://github.com/ivecanahi/simulacion-practicas/tree/main/Unidad 1/practica03**